

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Alocação Dinâmica

Alocação Dinâmica

- Conceito
- Mapa de memória de C
- O operador **sizeof**
- Funções de C para alocação dinâmica
- Acesso a elementos de uma struct via ponteiro
- Aplicação de alocação dinâmica

Conceito

Alocação Dinâmica

A declaração de uma variável em um programa indica que será necessário reservar (alocar) um espaço de memória para ela.

Esse espaço de memória é definido em tempo de compilação.

Por exemplo:

```
int a;  
float notas[45];  
struct paciente listaPacientes[500];
```

Alocação Dinâmica

Porém, há situações em que não se sabe antecipadamente o espaço que um dado ocupará na memória.

Por exemplo: em um programa para cadastrar pacientes, pode-se não saber antecipadamente quantos pacientes serão cadastrados.

Nessa situação, o espaço para o dado será reservado (alocado) durante a execução do programa.

Isso se trata de **alocação dinâmica**.

Alocação dinâmica é o meio pelo qual o programa obtém espaço em memória para um dado enquanto está em execução.

Mapa de Memória de C

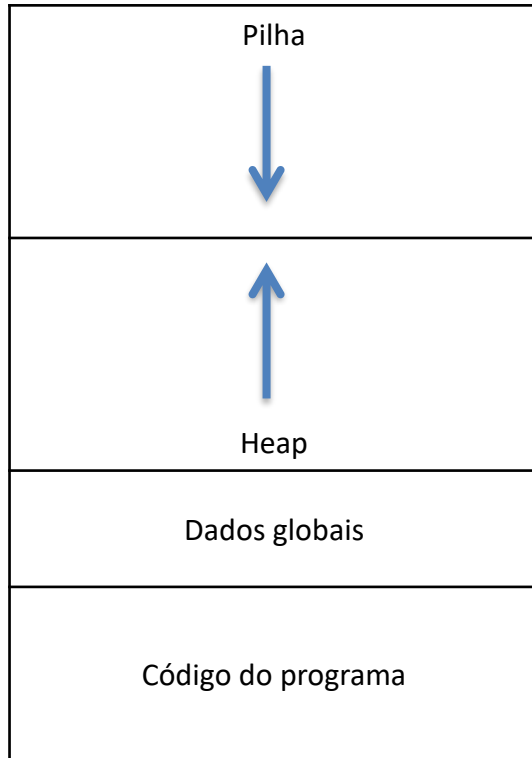
Mapa de Memória de C

Durante a execução de um programa em C, é reservada uma parte da memória para ele.

Esse espaço da memória é dividido em quatro regiões:

- código do programa
- dados globais
- pilha
- heap

Mapa de Memória de C



Pilha: armazena dados locais, dados de chamadas de funções, dentre outros, do programa.

Heap: região de memória livre para **alocação dinâmica** do programa.

Dados globais: armazena os dados globais do programa.

Código do programa: armazena o código executável do programa.

O operador `sizeof`

O operador **sizeof**

Para se fazer a alocação dinâmica, é necessário informar o tamanho do espaço necessário a ser alocado.

Para conhecer o tamanho de um dado, utiliza-se o operador **sizeof** .

A sintaxe de uso do operador é:

sizeof(dado)

O operador `sizeof`

```
#include <stdio.h>

typedef struct{
    char descricao[50];
    float preco;
    int qtd;
} Mercadoria;

void main(){
    int a;
    char palavra[10];

    printf("Tamanho de char: %d", sizeof(char));
    printf("\nTamanho de int: %d", sizeof(int));
    printf("\nTamanho de float: %d", sizeof(float));
    printf("\nTamanho de double: %d", sizeof(double));
    printf("\nTamanho da variavel a: %d", sizeof(a));
    printf("\nTamanho da variavel palavra: %d", sizeof(palavra));
    printf("\nTamanho de Mercadoria: %d", sizeof(Mercadoria));

}
```

Tamanho de char: 1
Tamanho de int: 4
Tamanho de float: 4
Tamanho de double: 8
Tamanho da variavel a: 4
Tamanho da variavel palavra: 10
Tamanho de Mercadoria: 60

Funções para Alocação Dinâmica

Funções para Alocação Dinâmica

As duas funções principais para alocação dinâmica são **malloc** e **free**.

void *malloc(size_t size)

Aloca uma região de memória no heap de tamanho `size` e devolve um ponteiro para o primeiro byte dessa região.

O ponteiro devolvido é do tipo `void`, que são ponteiros genéricos, ou seja, podem apontar para dado de qualquer tipo.

Caso não haja espaço suficiente no heap, a função devolve `NULL`.

Deve-se sempre verificar se o valor devolvido por `malloc` é diferente de `NULL`, pois a tentativa de usar um ponteiro nulo pode resultar numa quebra do sistema.

Funções para Alocação Dinâmica

void free(void *ptr)

Libera a área de memória apontada por `ptr`.

`ptr` deve ser um ponteiro que foi retornado por uma função que realizou alocação dinâmica (`malloc`, por exemplo). A utilização de um ponteiro inválido pode gerar erro no sistema.

Funções para Alocação Dinâmica

```
#include <stdio.h>
#include <stdlib.h>

typedef struct{
    char descricao[50];
    float preco;
    int qtd;
} Mercadoria;

void main(){

    Mercadoria *p;

    if ((p = malloc(sizeof(Mercadoria))) != NULL){
        printf("\nAlocacao realizada com sucesso.");
    }
    else{
        printf("\nErro de alocao de memoria. O programa será encerrado.");
        exit(1);
    }
}
```


Funções para Alocação Dinâmica

```
printf("\nO endereco da area alocada e %p", p);  
printf("\nSe a area nao e mais necessaria, devemos libera-la.");  
free (p) ;  
}
```

Saída do programa:

Alocacao realizada com sucesso.

O endereco da area alocada e 0000000000021490

Se a area nao e mais necessaria, devemos libera-la.

Funções para Alocação Dinâmica

Outras duas funções importantes são **calloc** e **realloc** .

void *calloc(size_t num, size_t size)

Aloca uma região de memória no heap de tamanho `num*size` e devolve um ponteiro para o primeiro byte dessa região.

Ou seja, aloca um espaço de memória para uma matriz de `num` elementos de tamanho `size`.

Assim, como `malloc`, `calloc` devolve um ponteiro do tipo `void`.

Funções para Alocação Dinâmica

`void *realloc(void *ptr, size_t size)`

Modifica o tamanho alocado anteriormente a `ptr` para o tamanho `size`.

`size` pode ser maior ou menor do que o espaço alocado anteriormente .

Devolve o ponteiro para a nova área alocada.

Acesso a Elementos de uma struct via Ponteiro

Acesso a Elementos de uma struct via Ponteiro

Para se acessar os elementos de uma struct via ponteiro, utiliza-se o operador ->

Exemplo:

Considere a struct Mercadoria do exemplo anterior. Ao fazer:

```
Mercadoria m, *p;
```

```
p = &m;
```

O dado `preco` da struct pode ser acessado de duas formas:

```
m.preco e p->preco
```

Acesso a Elementos de uma struct via Ponteiro

```
typedef struct{
    char descricao[50];
    float preco;
    int qtd;
} Mercadoria;

void main(){

    Mercadoria *p;
    Mercadoria m, *p1;
    p1=&m;

    if ((p = malloc(sizeof(Mercadoria))) != NULL){
        printf("\nAlocacao realizada com sucesso.");
    }
    else{
        printf("\nErro de alocao de memoria. O programa será encerrado.");
        exit(1);
    }
}
```

Acesso a Elementos de uma struct via Ponteiro

```
strcpy(p->descricao, "Notebook");  
p->preco = 4250,79;  
p->qtd = 100;  
  
printf("\nOs dados da Merdadoria sao: %s - %.2f - %d", p->descricao, p->preco,  
      p->qtd);  
  
strcpy(m.descricao, "Teste");  
p1->preco = 350.00;  
p1->qtd = 45;  
  
printf("\nOs dados da outra merdadoria sao: %s - %.2f - %d",  
      p1->descricao, p1->preco, m.qtd);  
  
}
```

Aplicação de Alocação Dinâmica

Aplicação de Alocação Dinâmica

A principal aplicação de alocação dinâmica é a construção de estruturas de dados dinâmicas como listas, pilhas e filas.

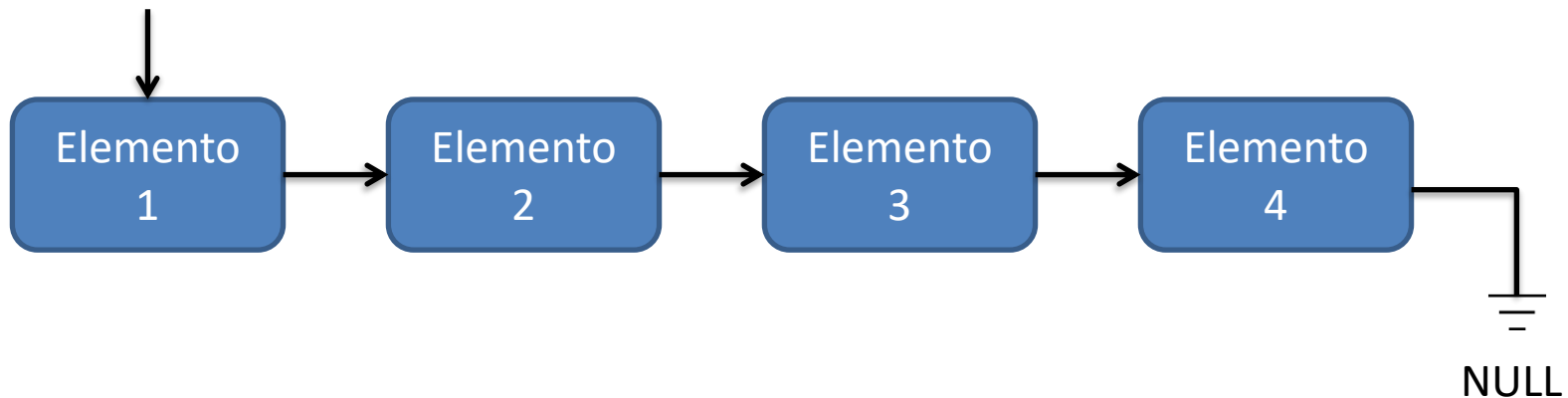
Esses são assuntos abordados de forma aprofundada na disciplina de Algoritmos e Estruturas de Dados I.

Lista

Uma **lista** é um conjunto de dados de um mesmo tipo que são encadeados, ou seja, um elemento **aponta** para o próximo.

O último elemento da lista não possui elemento a apontar.

Ponteiro para o
início da lista



Lista

Ao ser criada, a lista está vazia, ou seja, o “ponteiro para o início da lista” é nulo.

Ponteiro para o
início da lista

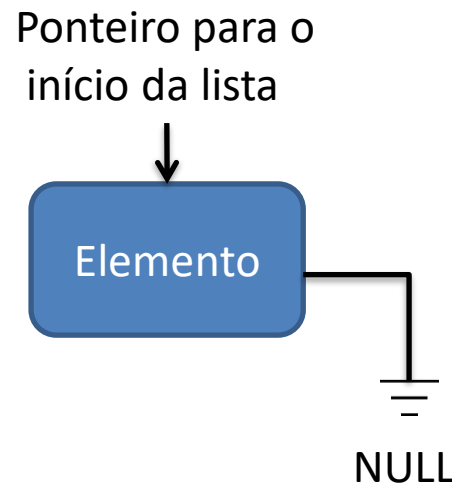


NULL

Lista

Para a inclusão do primeiro elemento, deve-se:

1. **Alocar** o novo elemento, fazendo apontar para nulo.
2. Fazer o “ponteiro para o início da lista” apontar para o novo elemento.

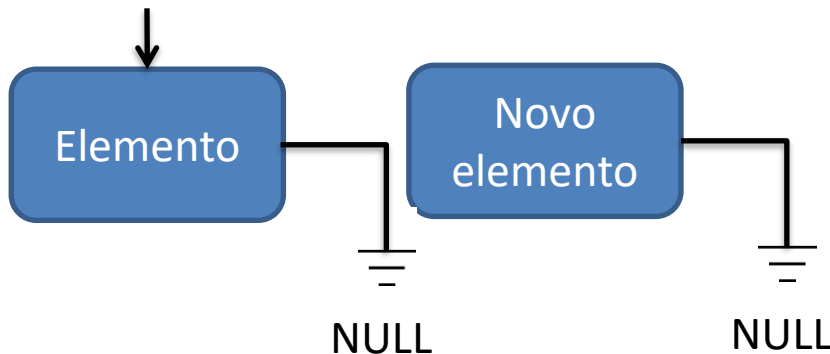


Lista

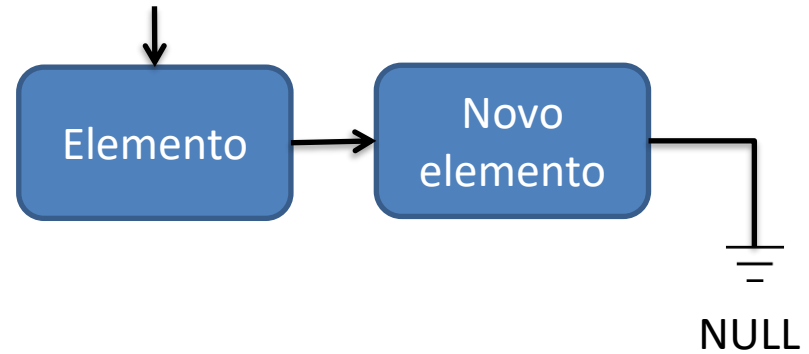
Para a inclusão de um novo elemento ao final da lista, deve-se:

1. **Alocar** o novo elemento, fazendo-o apontar para nulo.
2. Fazer o último elemento da lista apontar para o novo elemento

Ponteiro para o
início da lista



Ponteiro para o
início da lista



Lista

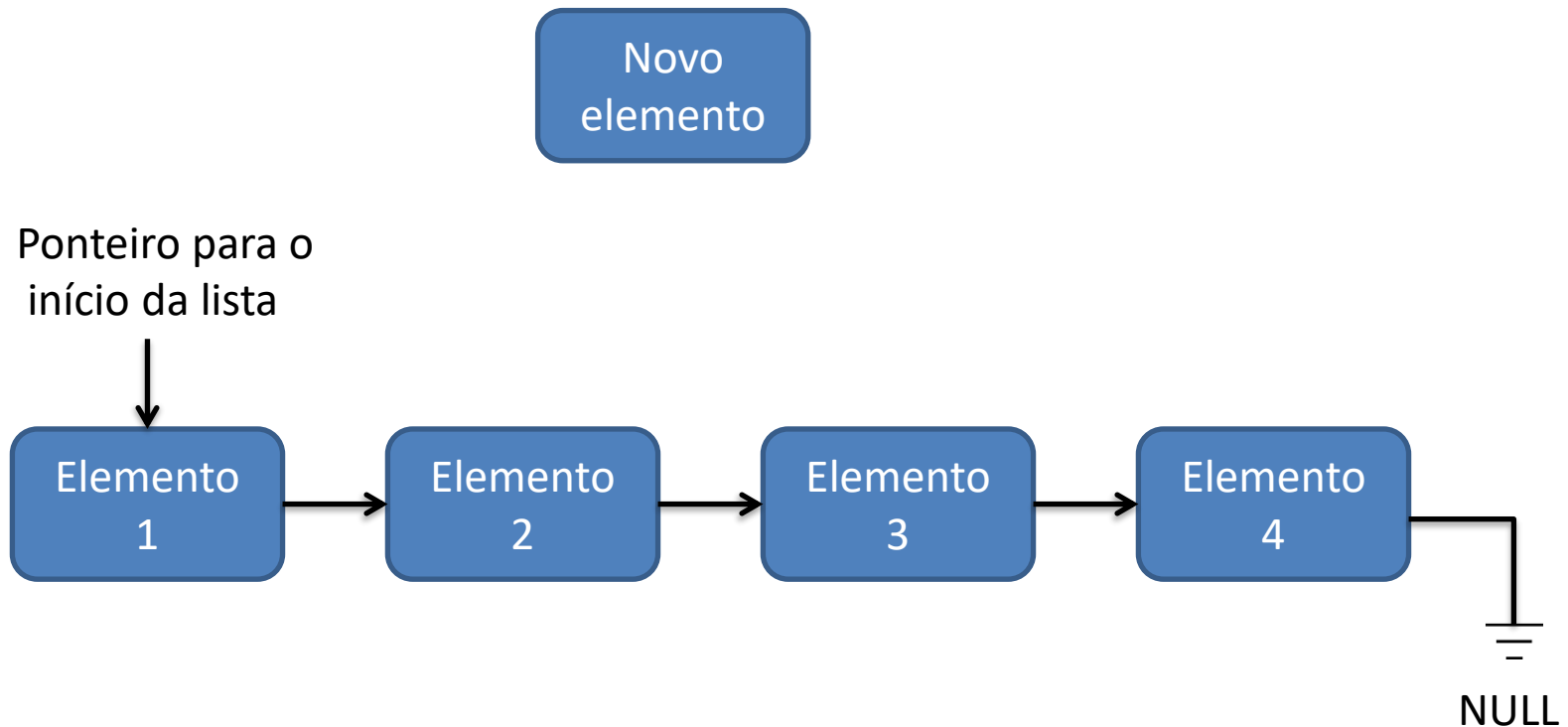
Na lista, pode-se incluir ou excluir elemento em qualquer posição.

Suponha que se queira incluir um novo elemento entre os elementos 2 e 3. Para isso, é necessário:

1. **Alocar** o novo elemento.
2. Fazê-lo apontar para o Elemento 3.
3. Fazer o Elemento 2 apontar para o novo elemento.

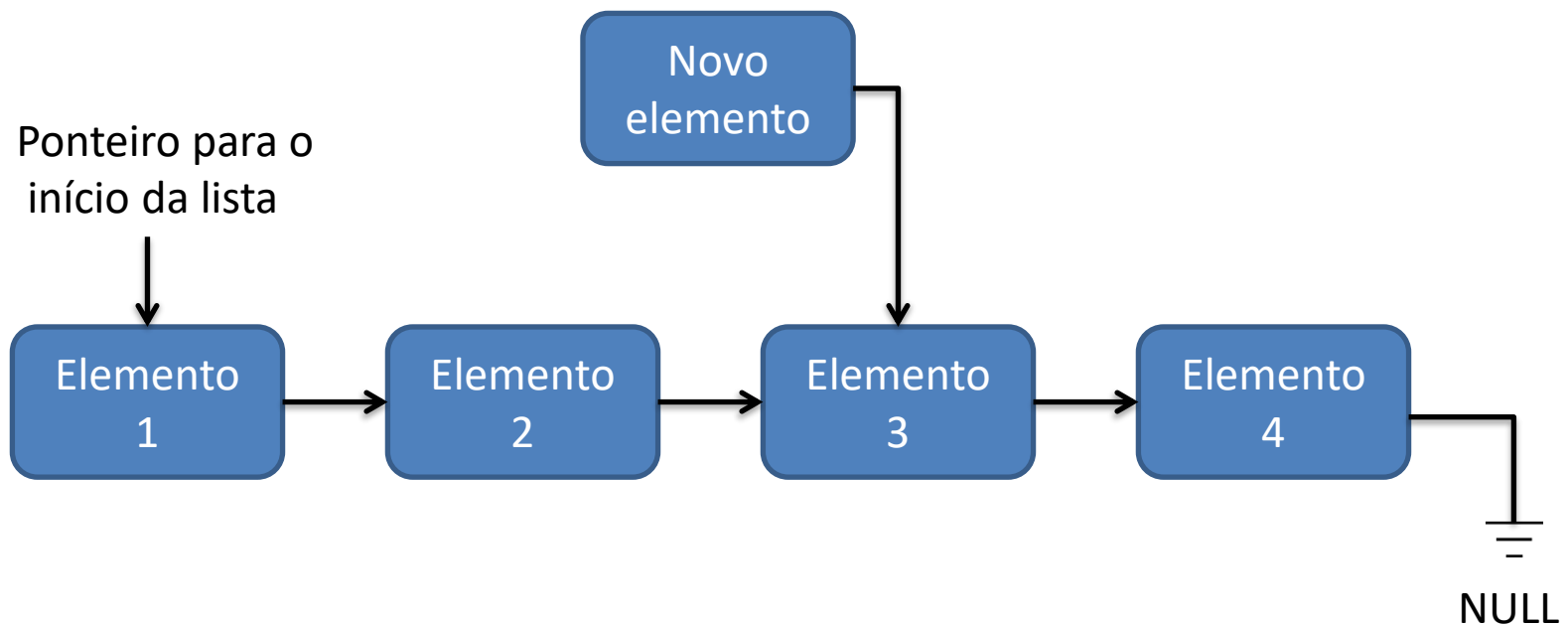
Lista

1. **Alocar** o novo elemento.



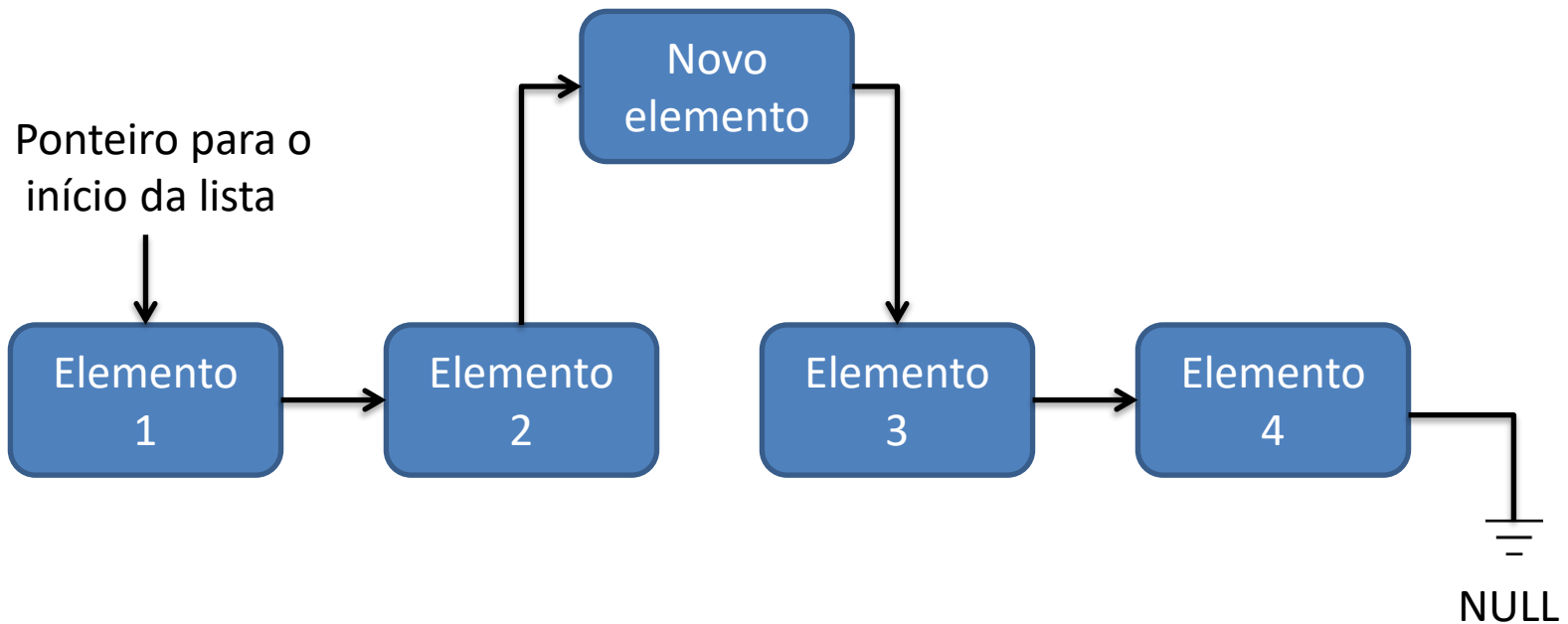
Lista

2. Fazê-lo apontar para o Elemento 3.



Lista

3. Fazer o Elemento 2 apontar para o novo elemento.



Lista

Suponha que o elemento da Lista seja do tipo pessoa. Para se implementar a Lista, cada elemento deve possuir um ponteiro para o próximo, que também é do tipo pessoa.

```
struct pessoa{  
    char nome[50];  
    int idade;  
    float altura;  
    struct pessoa *proximo;  
};
```

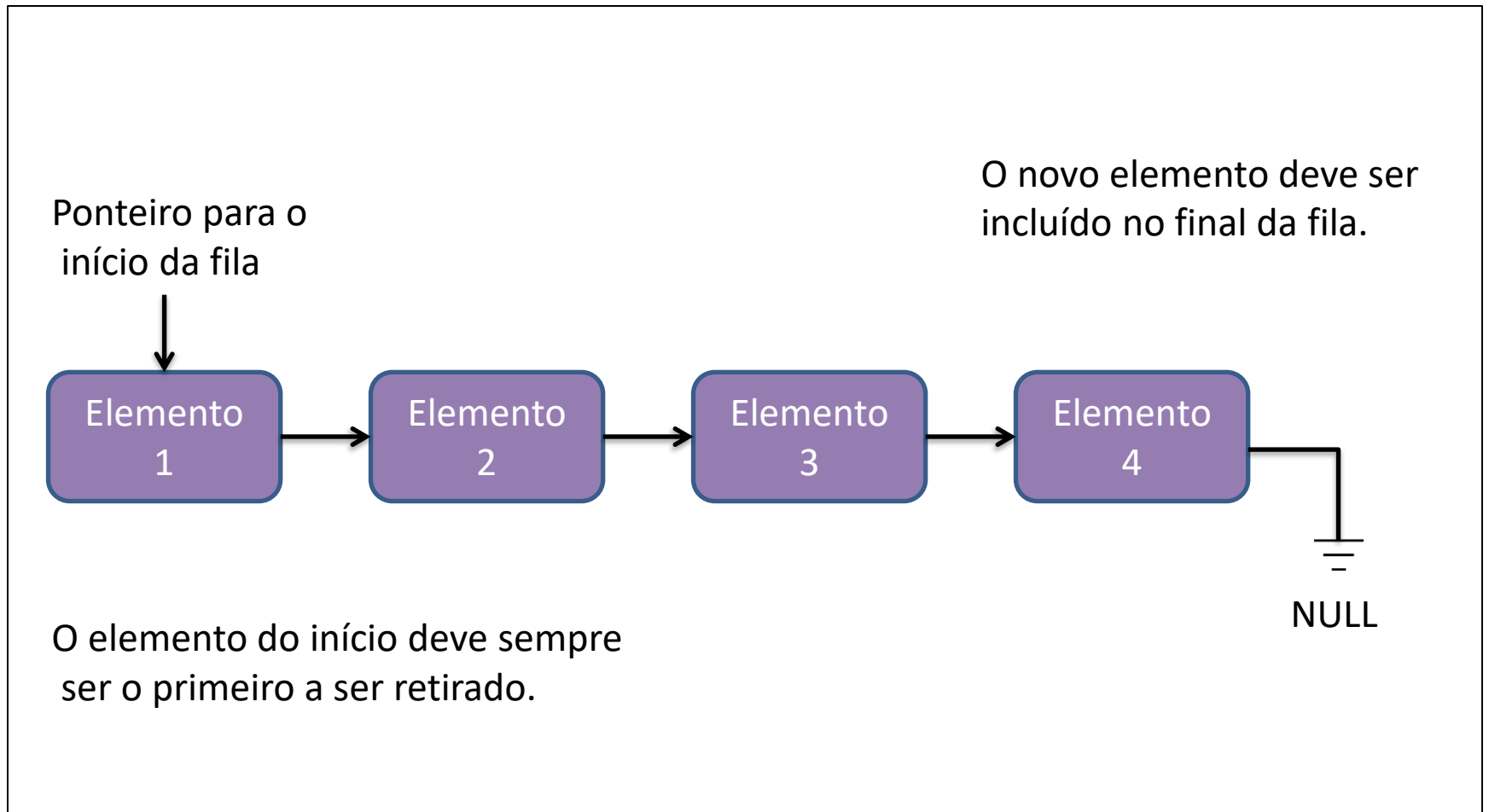
Fila

Uma **fila** é uma estrutura de dados construída de forma similar à lista, porém com as seguintes restrições:

O novo elemento deve sempre ser incluído ao **final** da fila.

O **primeiro** elemento incluído é o primeiro elemento a ser removido da **fila**.

Fila



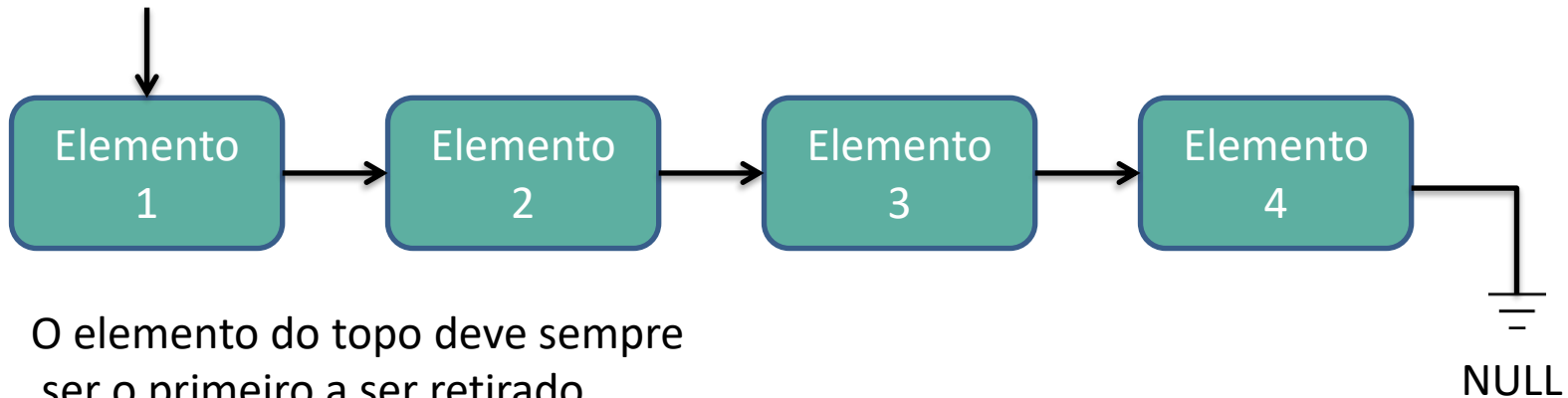
Pilha

Uma **pilha** é uma estrutura de dados construída de forma similar à lista, porém com as seguintes restrições:

- O novo elemento deve sempre ser incluído **no início** (topo) da pilha.
- O **último** elemento incluído é o **primeiro** elemento a ser removido da pilha.

Pilha

Ponteiro para o
topo da pilha



O elemento do topo deve sempre
ser o primeiro a ser retirado

O novo elemento deve ser
incluído no topo da pilha.

Aplicações das Estruturas de Dados

As aplicações de estruturas de dados como lista, fila e pilha são inúmeras em programação.

Exemplos:

- Programa para gerenciar atendimentos de chamadas telefônicas

Esse problema é do tipo “o primeiro a entrar deve ser o primeiro a sair”. Portanto, aplica-se fila.

- A funcionalidade “desfazer” do editor de texto

A última ação realizada pelo usuário deve ser a primeira a ser desfeita. Esse problema é do tipo “o último a entrar deve ser o primeiro a sair”. Portanto, aplica-se pilha.